

BACKEND ENGINEER

한재혁

Han Jaehyuk

주식회사 텐핑거스 · 2025.10~ · 서울

복잡한 운영 문제를 믿을 수 있는 제품 경험으로 바꿉니다.

Python 백엔드부터 데이터 정합성·성능·MSA 전환까지,
실제 사용자 경로를 끝까지 따라갑니다.

MSA migration

9개 API

클라이언트 변경 없이
일부 운영 트래픽 전환

API response

< 1초

30초 이상 응답 실패 해소
운영 주로 300~500ms

Data loaded

1/1,000

찜 목록 응답 계산에 읽던 행
같은 응답 기준 전후 비교

Data integrity

4건

복제 지연 · 한도 오염 ·
재계산 동시성
잔액 회수 경계를 순차 해결

jh.jaehyuk213@gmail.comgithub.com/Jh-jaehyukjh-jaehyuk.github.io/portfolio

클라이언트 계약을 깨지 않고 구독 기능을 전용 서비스로 옮겼습니다.

Production	API 9개
Client changes	0
Status	일부 전환 완료

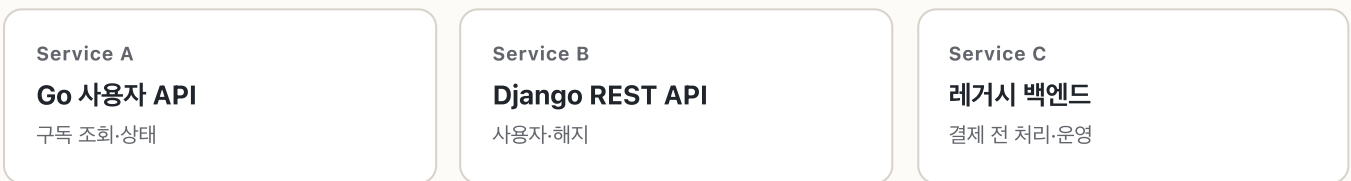
문제

하나의 구독 사용자와 상태를 세 레거시 백엔드가 서로 다른 계약과 예외 처리로 다루고 있었습니다. 작은 변경도 영향 범위를 가늠하기 어려웠습니다.

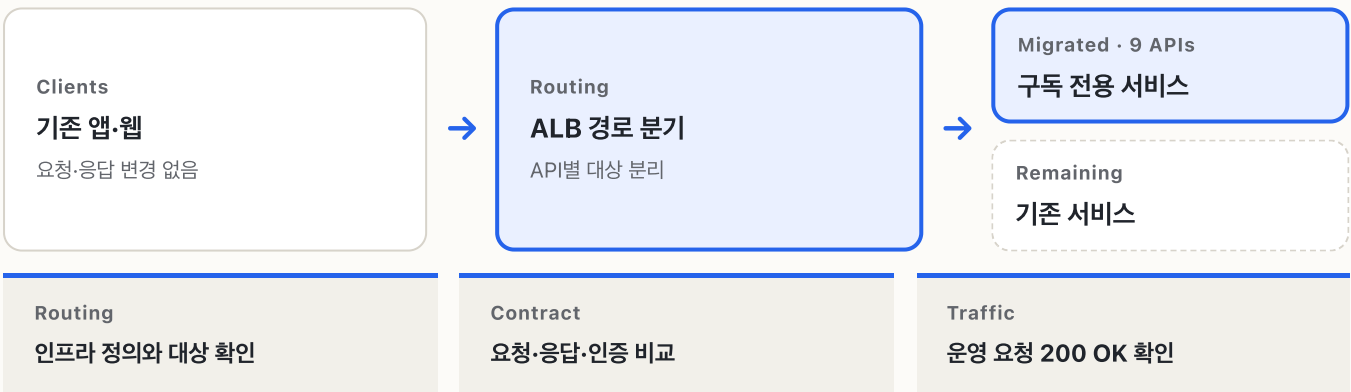
제가 맡은 범위

이전 대상을 좁히고 호출 경로와 계약을 비교한 뒤, 구독 전용 서비스의 API를 직접 구현했습니다. 인프라 변경은 권한 보유자가 적용했고, 배포 후 라우팅·운영 로그 확인도 제가 맡았습니다.

BEFORE 하나의 도메인, 세 개의 실행 경로



AFTER 클라이언트 계약을 유지한 점진적 전환



도구와 판단의 분담

세 저장소에 걸친 호출 경로와 계약 차이는 Codex·Claude Code로 좁혔습니다. 전용 서비스 API 구현과 이전 범위·위험 판단, 운영 검증은 직접 맡았습니다.

30초 넘게 타임아웃되던 응답을 1초 미만으로 줄였습니다.

Before	30초+ timeout
Production	300~500ms
Rows loaded	약 1/1,000

측정 방법과 범위

찜 목록 매장의 결제 이력은 편차가 극단적이라 특정 매장을 찜한 사용자에게서만 재현됐고, 평균적인 목록으로는 보이지 않았습니다.

응답 시간은 같은 계정·데이터로 Postman에서 전후를 반복 요청해 확인했고, 데이터 접근량은 전후 코드를 같은 합성 데이터에 올려 같은 응답임을 확인한 뒤 쿼리 수와 읽은 행 수를 셉니다. 공개 자료에는 배수만 실습니다.

Before · 미리 읽고, 또 조회

Prefetch

결제 원본 통째로 적재

결제 많은 매장일수록 급증



Per item

Serializer·도메인 메서드

선조회를 못 쓰고 관계마다 재조회



Result

30초 이상 · 요청 실패

After · 건어내고 DB에 맡김

Query plan

원본 적재 제거·DB 집계

판매량을 Subquery로



Serializer

선조회한 잔액을 실제로 재사용

재조회 대신 읽은 값으로 계산



Result

모든 측정 1초 미만

읽은 행 약 1/1,000·쿼리 약 1/3

01

소비 경로 추적

View에서 멈추지 않고 serializer와 도메인 메서드까지 확인했습니다.

02

원본 대신 집계값

결제 원본을 읽던 선조회를 건어내고 판매량은 Subquery로 DB에서 집계했습니다.

03

선조회가 닿게

도메인 메서드가 재조회하는 대신 이미 읽어 둔 값을 쓰도록 바꿨습니다.

검증 단계	조건	확인 결과
수정 전	동일 계정·데이터	30초 이상 지연 후 요청 실패
로컬·스테이징	수정 코드 반복 요청	1초 미만·목록 데이터 확인
운영·앱	배포 직후 반복 요청	주로 300~500ms·목록 화면 복구
데이터 접근량	수정 전후 코드·동일 응답	읽은 행 약 1/1,000·쿼리 약 1/3

사용량이 약 4배 늘어난 뒤 드러난 리뷰 보상 정합성 문제를 해결했습니다.

Project context

사용량 약 4배

Comparison

동일 조건 7일

My scope

정합성 경로

CORE INVARIANT

같은 리뷰 상태라면 여러 번 다시 계산해도
최종 잔액과 이력은 한 번 계산한 결과와 같아야 해요.

01 Writer 기준 조회

복제 지연이 보상 판단을 바꾸지 않도록 최신 상태를
writer에서 읽었습니다.

02 현재 리뷰 제외

수정 중인 리뷰가 자신의 보상 한도 계산에 다시 포함되지
않게 했습니다.

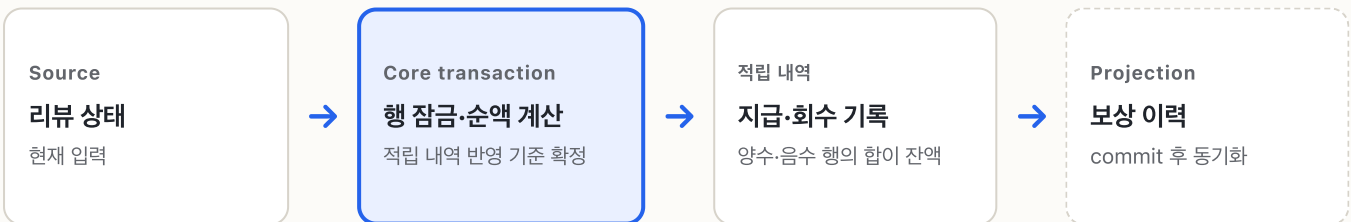
03 행 잠금과 이력 교체

동시 재계산이 중복 결과를 만들지 않도록 transaction
경계를 정리했습니다.

04 음수 적립 행

취소 상태만 남기지 않고 실제 회수액을 행으로 기록했습니다.

CROSS-STORE BOUNDARY 적립 내역 commit 성공을 기준으로 동기화



구분	확인된 결과	설명
프로젝트 성과	사용량 약 4배	동일 조건의 7일 비교, 프로젝트 전체 결과
개인 기여	수정·재계산·회수 정합성	Writer read, 현재 리뷰 제외, 행 잠금, 음수 적립 행
운영 안전망	재실행 가능한 보정	적립 내역 commit 이후 동기화와 일일 reconciliation

좋은 글이 만든 구매를 추적하고 작성자에게 보상했습니다.

Rule

마지막 링크

Scope

백엔드 전 구간

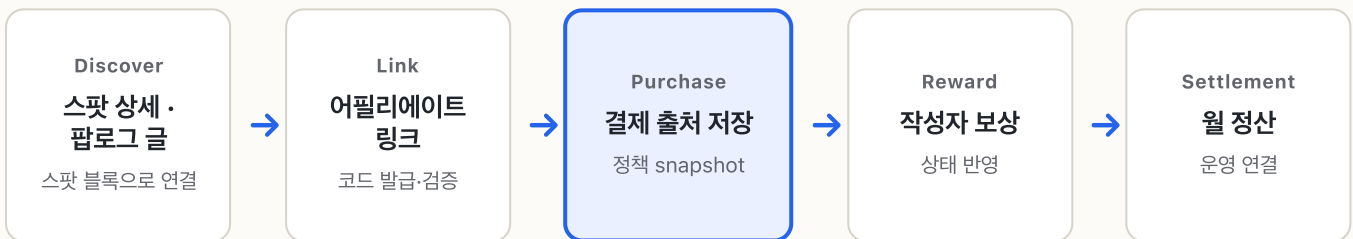
제품 맥락

팝로그는 유저가 서비스 안에 쓰는 블로그 글입니다. 글에 스팟 블록을 넣으면 그 스팟 상세 페이지에도 글이 노출돼, 스팟을 보러 온 사람이 글을 만나고 구매로 이어지는 경로가 생깁니다. 그 구매를 작성자에게 돌려주려 했지만 기여를 추적할 방법이 없었습니다.

제가 맡은 범위

링크 생성·검증부터 웹과 앱의 코드 전달, 결제 출처 저장, 전환 상태 반영, 작성자 보상과 월 정산까지 백엔드 전 구간을 구현했습니다.

PIPELINE 콘텐츠에서 월 정산까지 하나의 흐름



KEY RULES 정책이 바뀌어도 과거 구매의 의미를 보존

01 · Window

7일 이내 링크만 인정

구매 시점에 유효성을 판단해 출처를 확정했습니다.

02 · Priority

마지막 유효 링크

여러 콘텐츠를 거쳤을 때 가장 최근 기여를 사용했습니다.

03 · Snapshot

구매 당시 정책 저장

이후 정책이 바뀌어도 기존 정산 결과가 달라지지 않게 했습니다.

같은 결제로는 한 번만

결제 알림은 재전송될 수 있고 보정 작업도 다시 도니까, 전환 기록에 출처·결제 식별자·전환 종류로 유일 제약을 걸어 몇 번을 실행해도 한 건만 남게 했습니다. 기록하는 사이에 결제가 취소되는 경우를 위해 기록 직후 취소 여부를 한 번 더 확인하고, 취소는 삭제 대신 시각·사유로 남겼습니다.

결과

보이지 않던 콘텐츠의 기여를 결제 데이터에 남겨 작성자 보상과 월 정산으로 연결했습니다. 공개 자료에는 민감한 원수치를 제외하고, 추적·정산 구조와 확인 가능한 결과만 담았습니다.

매달 한 번 무료로 받는 마케팅을 사장님이 직접 운영하게 만들었습니다.

Benefit

월 1회 무료

My focus

상태·응답 계약

Status

앱 연동·QA

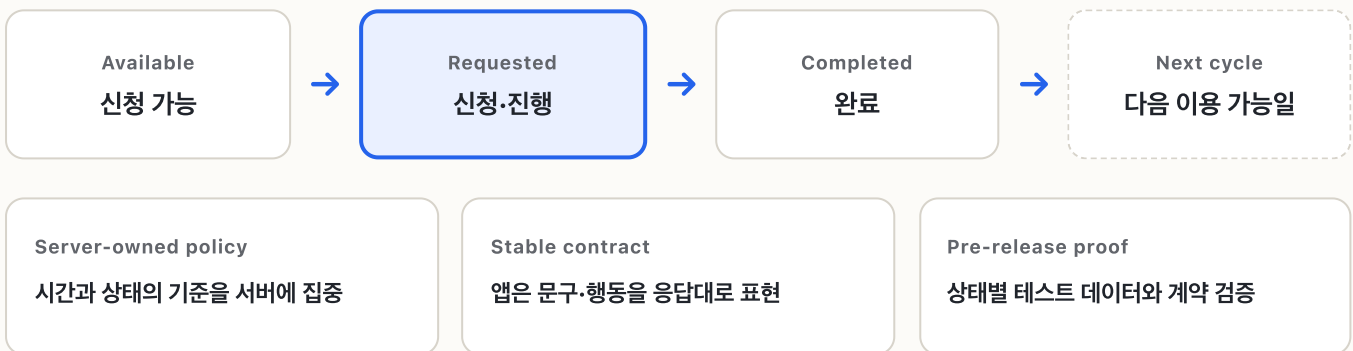
제품 목표

사장님 앱 리뉴얼에서 제휴 매장이 매달 한 번 무료 마케팅을 받을 수 있다는 점을 경쟁 포인트로 만들었습니다.

제가 맡은 범위

백엔드 구현과 진행 상태 계산, 응답 계약, 운영 규칙과 검증 흐름을 맡았습니다. 핵심 도메인 모델과 캠페인 생성은 팀원과 함께 개발했습니다.

STATE MODEL 사용자 관점 4구간 · 내부 7상태



월 1회를 동시 요청에서도

조회하고 만드는 사이에 요청이 두 번 들어오면 같은 달에 두 건이 생깁니다. 매장 단위 잠금을 먼저 잡고, 트랜잭션 안에서 기존 캠페인 행까지 잠근 뒤에 검사하도록 했습니다.

현재 상태

백엔드 구현과 상태 계약은 완료했고, 앱 연동과 QA를 진행하고 있습니다. 아직 출시 성과를 말할 단계는 아니라 구조와 검증 방식만 담았습니다.

제품의 약속이 운영에서도 지켜지게 만듭니다.

WORKING PRINCIPLES

실제 경로를 끝까지 따라갑니다.

화면부터 API·비동기 작업·데이터 저장까지 확인합니다.

운영에서 한 번 더 확인합니다.

배포가 아니라 실제 동작을 완료 기준으로 삼습니다.

AI의 속도에 판단을 더합니다.

분석과 초안은 빠르게, 설계와 결과는 직접 책임집니다.

CORE STACK

Python

Django / DRF

Litestar

FastAPI

Celery

MySQL

Redis

Docker

AWS ECS · RDS · ALB

GitHub Actions

함께 풀고 싶은 문제가 있다면 편하게 연락 주세요.

jh.jaehyuk213@gmail.com

github.com/Jh-jaehyuk

jh-jaehyuk.github.io/portfolio